

Article

Performance Evaluation of MongoDB and RavenDB in IIoT-Inspired Data-Intensive Mobile and Web Applications

Mădălina Ciumac ¹, Cornelia Aurora Győrodi ^{2,*}, Robert Ștefan Győrodi ^{2,*} and Felicia Mirabela Costea ²¹ Faculty of Electrical Engineering and Information Technology, University of Oradea, 410087 Oradea, Romania; mada.ciumac@gmail.com² Department of Computers and Information Technology, University of Oradea, 410087 Oradea, Romania; mira_costea@uoradea.ro

* Correspondence: cgyorodi@uoradea.ro (C.A.G.); rgyorodi@uoradea.ro (R.Ș.G.)

Abstract

The exponential growth of data generated by modern digital applications, including systems inspired by Industrial Internet of Things (IIoT) requirements, has accelerated the adoption of NoSQL databases due to their scalability, flexibility, and performance advantages over traditional relational systems. Among document-oriented solutions, MongoDB and RavenDB stand out due to their architectural features and their ability to manage dynamic, large-scale datasets. This paper presents a comparative analysis of MongoDB and RavenDB, focusing on the performance of fundamental CRUD (Create, Read, Update, Delete) operations. To ensure a controlled performance evaluation, a mobile and web application for managing product orders was implemented as a case study inspired by IIoT data characteristics, such as high data volume and frequent transactional operations, with experiments conducted on datasets ranging from 1000 to 1,000,000 records. Beyond the core CRUD evaluation, the study also investigates advanced operational scenarios, including joint processing strategies (lookup versus document inclusion), bulk data ingestion techniques, aggregation performance, and full-text search capabilities. These complementary tests provide deeper insight into the systems' architectural strengths and their behavior under more complex and data-intensive workloads. The experimental results highlight MongoDB's consistent performance advantage in terms of response time, particularly with large data volumes, while RavenDB demonstrates competitive behavior and offers additional benefits such as built-in ACID compliance, automatic indexing, and optimized mechanisms for relational retrieval and bulk ingestion. The analysis does not propose a new benchmarking methodology but provides practical insights for selecting an appropriate document-oriented database for data intensive mobile and web application contexts, including IIoT-inspired data characteristics, based on a controlled single-node experimental setting, while acknowledging the limitations of a single-host experimental environment.



Academic Editors: Paolo Bellavista, Zhihao Liu, Franco Davoli and Davide Borsatti

Received: 16 November 2025

Revised: 13 January 2026

Accepted: 16 January 2026

Published: 20 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: NoSQL; data-intensive applications; MongoDB; RavenDB; CRUD operations

1. Introduction

The exponential growth of data generated by modern digital systems has created challenges in data storage, retrieval, and processing. Traditional relational database management systems (RDBMS), while reliable and mature, often encounter limitations when dealing with unstructured, semi-structured, or rapidly evolving data models [1]. To address these challenges, NoSQL databases have emerged as a robust alternative,

offering schema flexibility, horizontal scalability, and improved performance in distributed environments [2,3].

Among the various categories of NoSQL systems, document-oriented databases have gained particular relevance due to their ability to represent hierarchical and heterogeneous data structures in a natural and developer-friendly format [4,5]. Their flexibility and adaptability make them well-suited for modern application domains such as e-commerce platforms, customer relationship management (CRM) systems, recommendation engines, and social media analytics, where personalization, real-time insights, and fault tolerance are essential [6,7].

MongoDB and RavenDB, two leading document-oriented databases, offer robust solutions for enterprises seeking scalable and high-performance data management. MongoDB, introduced in 2009, is one of the most widely adopted NoSQL databases, recognized for its scalability, flexible schema design, and strong community support [8]. RavenDB, launched in 2010, distinguishes itself through its native support for ACID transactions, integrated full-text search, and tight integration with the .NET ecosystem, making it particularly appealing for enterprise applications [9,10]. While both systems share a document-based architecture, they differ significantly in their approaches to indexing, query models, replication strategies, and consistency guarantees [11].

This paper contributes to the ongoing debate on the suitability of document-based databases by presenting a comparative study of MongoDB and RavenDB, conducted through a case study mobile application for managing product orders. The focus is on evaluating CRUD (Create, Read, Update, Delete) operations under conditions of varying data volumes to highlight the trade-offs between performance, scalability, and consistency. The results aim to provide practical guidance to developers and researchers in selecting the most suitable document-oriented database for big data applications.

In this study, we focus on MongoDB and RavenDB because both systems are widely adopted in production-grade mobile, web, and IIoT architectures inspired by IIoT requirements, yet exhibit fundamentally different architectural philosophies. Rather than offering an exhaustive comparison of all document-oriented databases, the objective is to perform an in-depth evaluation of two mature, representative platforms, emphasizing how their indexing strategies, consistency models, caching mechanisms, and query execution pipelines impact real-world workloads in IIoT-oriented mobile applications. This level of focus enables a clearer and more meaningful interpretation of architectural trade-offs relevant to practitioners.

The paper offers a substantial contribution by providing a direct and comprehensive experimental comparison between two document-oriented databases, MongoDB and RavenDB, emphasizing the key trade-offs between performance, scalability, and consistency. The results show that MongoDB delivers superior raw performance and lower response times for large data volumes, reflecting its architecture optimized for flexibility and horizontal scaling. In contrast, RavenDB ensures greater robustness and consistency through its native ACID transactional model and automatic indexing mechanisms. In addition to the core CRUD evaluation, the study incorporates a wider set of advanced tests—including join processing strategies, high-throughput bulk ingestion, aggregation efficiency, and full-text search performance—offering deeper insight into how architectural design choices influence system behavior under both simple and complex workloads. The research provides valuable practical guidance for selecting appropriate NoSQL solutions in IIoT, mobile, or enterprise environments, depending on specific requirements regarding ingestion speed, consistency guarantees, analytical needs, and transactional reliability.

This study does not aim to introduce new benchmarking methodologies. Instead, its contribution lies in a systematic and reproducible experimental comparison of two mature

document-oriented databases—MongoDB and RavenDB—focusing on how architectural design choices influence performance across representative workloads. By providing detailed configuration, repeated measurements, and workload-specific analysis, the paper offers practical insights for researchers and practitioners selecting NoSQL solutions for data-intensive applications.

Unlike standardized industry benchmarks, such as those defined by the Transaction Processing Performance Council (TPC), which are primarily designed for relational database systems, this study adopts an application-driven workload tailored to document-oriented NoSQL databases. This approach allows the evaluation of database behavior under representative document-centric access patterns and architectural features.

2. Related Work

The growing demand for scalable and flexible data management solutions has led to extensive research on NoSQL systems. In [1], Elgendy and Elragal highlight the role of big data analytics as a driving force for the adoption of non-relational technologies. In [2] the authors provide an analysis of NoSQL databases, highlighting schema flexibility, scalability, and suitability for heterogeneous data. Subsequent work, such as that of Cattell [3] and Grolinger et al. [12], has further highlighted the architectural trade-offs of NoSQL systems compared to SQL, especially in cloud environments, where elasticity and replication are crucial.

More recent contributions analyze the challenges posed by modern database technologies. Miryala [6] identifies emerging trends in the adoption of NoSQL, highlighting the challenges between performance and consistency. Martins et al. [7] describe a real-world use case for NoSQL in practice, emphasizing the importance of matching database capabilities to application-specific requirements. Gillenson [13] contextualizes these developments within the broader evolution of database management systems, emphasizing the paradigm shift brought about by document-oriented approaches.

Experimental studies also provide valuable insights into document-oriented NoSQL databases. Carvalho et al. [4] evaluated document-oriented systems such as Couchbase, CouchDB, and MongoDB, reporting measurable differences in CRUD performance. In [5], Tudorica and Bucur provided benchmarks for several NoSQL databases. However, these studies either omit RavenDB entirely or do not provide a direct comparison with MongoDB. More recent, focused analyses, such as those presented by Dhanagari in [14] on MongoDB's consistency mechanisms and Qaddara et al. in [15] on SQL vs. NoSQL in parallel processing, confirm the growing interest in rigorous performance evaluation. This confirms the breadth of research on NoSQL systems, but also reveals an imbalance: MongoDB is frequently tested as a benchmark, while RavenDB is often overlooked in academic studies, despite being a mature, document-oriented database.

This difference highlights the need for systematic comparisons between MongoDB and RavenDB that allow software developers and researchers to make informed decisions that balance raw performance, transactional integrity, and integration requirements.

3. Features of MongoDB and RavenDB

MongoDB is one of the most widely adopted NoSQL databases, consistently maintaining its position as a preferred choice for handling large-scale, unstructured data since its introduction in 2009 [8]. It is a cross-platform, open-source database that employs a document-based model, storing data in BSON (Binary JSON) format. This schema-free design enables developers to handle dynamic and evolving data structures effectively. MongoDB offers high performance and availability through horizontal scaling with sharding, as well as robust aggregation frameworks for complex data operations [9,12]. These features

have made it a popular choice for applications requiring rapid scalability and flexibility. RavenDB, on the other hand, is a distributed NoSQL database launched in 2010. It also follows a document-oriented model but emphasizes ease of use, built-in full-text search, and ACID (Atomicity, Consistency, Isolation, Durability) compliance, distinguishing it from many other NoSQL databases. RavenDB excels in scenarios where data consistency and real-time indexing are critical [11]. Its real-time performance benefits applications requiring immediate insights from incoming data streams [10]. RavenDB's indexing capabilities allow developers to define indexes proactively or rely on the database to automatically create optimal indexes for queries, improving efficiency [11].

A. Data Model

MongoDB utilizes BSON objects, where each document consists of key-value pairs identified by a unique key. This flexible model does not require a predefined schema, allowing developers to dynamically modify document structures as application needs evolve [3,14]. Similarly, RavenDB employs JSON documents but enhances the model with advanced indexing capabilities to support efficient query execution [10,11].

B. Query Model

MongoDB supports JSON-like queries using its 'find' method and offers batch processing and aggregation through features like MapReduce. Additionally, the '\$lookup' operator enables Left Outer Join operations between collections, improving the handling of data relationships [5]. RavenDB, on the other hand, uses LINQ (Language-Integrated Query) for querying, making it especially appealing to .NET developers. Its query engine supports complex queries with full-text search, dynamic aggregation, and result caching [11].

C. Scalability and Replication

MongoDB enables horizontal scaling through sharding, which partitions datasets across multiple nodes. It also supports replica sets to ensure high availability and redundancy, with automatic failover handling [3]. In contrast, RavenDB offers built-in clustering capabilities that replicate data across nodes while preserving ACID compliance. This approach ensures both consistency and availability, making it well-suited for mission-critical applications [11].

D. Security Features

Both MongoDB and RavenDB place a strong emphasis on security, offering comprehensive features to safeguard sensitive data. MongoDB implements role-based access control (RBAC), encryption, and TLS/SSL to ensure secure communication. Similarly, RavenDB includes encryption-at-rest, certificate-based authentication, and extensive auditing capabilities to support secure and accountable data management [14].

E. Cost and Licensing

MongoDB provides a free Community Edition in addition to its enterprise and cloud-based solutions, which require subscriptions for access to advanced features and professional support [8,16]. RavenDB adopts a comparable approach, offering a free version suitable for small-scale applications, while its paid licenses cater to larger deployments, striking a balance between accessibility for individual developers and scalability for enterprise needs [10,17]. Table 1 summarizes the characteristics of the two NoSQL databases, MongoDB and RavenDB.

Table 1. MongoDB and RavenDB Features.

Feature/Criteria	MongoDB	RavenDB
Release year	2009 [8]	2010 [10]
Data model	BSON documents (binary JSON), schema-less	JSON documents, schema-less, enhanced with advanced indexing
Query model	JSON-like queries, aggregation framework, MapReduce, \$lookup for joins	LINQ queries, built-in full-text search, dynamic aggregation, caching
Indexing	Manual or automatic index creation, compound and geospatial indexes	Automatic and user-defined indexes, optimized for .NET queries
Transactions	Multi-document transactions (since v4.0), not ACID by default	Native ACID transactions across documents and collections
Scalability	Sharding (horizontal partitioning), replica sets for HA	Clustering with replication and failover, ACID consistency preserved
Performance	Lower response times on CRUD with large datasets	Slightly higher latency, but competitive even at high volumes
Integration	Cross-platform, strong ecosystem, widely used in cloud and big data stacks	Tight integration with .NET ecosystem (C#, LINQ), optimized for enterprise
Security	Role-based access control (RBAC), TLS/SSL, encryption-at-rest (Enterprise)	Encryption-at-rest, certificate-based authentication, extensive auditing
Licensing and Cost	Free Community Edition, Enterprise and Atlas (cloud) require subscription	Free tier available; commercial licenses needed for larger deployments
Use cases	Big data analytics, e-commerce, social media, IoT	Enterprise apps requiring ACID, .NET apps, real-time analytics with indexing

Architectural Design and Performance Implications

Understanding the internal design philosophies of MongoDB and RavenDB is essential for interpreting the experimental results presented in this study. Although both systems follow a document-oriented model, they differ substantially in their storage engines, consistency guarantees, indexing strategies, and caching mechanisms—factors that directly influence the performance patterns observed in Section 5.

Write and logging model.

MongoDB employs the WiredTiger [8,9] storage engine, which implements document-level locking to allow concurrent execution of operations on multiple documents within the same collection. This minimizes deadlocks and increases throughput for insert and update operations. WiredTiger writes data to memory first and performs persistence to disk through periodic checkpointing. Furthermore, the asynchronous implementation of the write-ahead log significantly reduces commit latency. These optimizations explain the lower response times achieved by MongoDB in INSERT and UPDATE operations, particularly for large datasets.

In contrast, RavenDB [10] uses a synchronous transactional journaling mechanism, confirming each write operation on disk before returning the response to the application. This guarantees full ACID durability but introduces additional latency due to I/O synchronization. While RavenDB provides document-level isolation, the commit is applied synchronously to the entire transaction context, increasing data safety at the expense of speed.

Consistency and distributed data management.

MongoDB adopts an eventual consistency model based on asynchronous replication within replica sets. Data propagation occurs in the background, allowing the system to respond quickly even when replicas are not fully synchronized. This design improves response time and scalability but may temporarily expose clients to stale reads immediately after write operations. RavenDB [10], on the other hand, enforces strong consistency by applying a cluster-wide consensus mechanism with synchronous replication. Once

a transaction is confirmed, all nodes hold the same data state, ensuring correctness in exchange for a moderate increase in latency.

Indexing mechanisms.

MongoDB [8] uses manually defined B-Tree or B+Tree indexes, providing developers with precise control over query optimization and minimizing the overhead on write operations through incremental index updates. If no index is defined, queries are executed via collection scan, maintaining high write throughput.

RavenDB employs an automatic and adaptive indexing system that observes query patterns and creates or updates indexes in the background. This approach enhances read performance and reduces administrative effort but adds a small cost to insert and update operations, as the system continuously determines whether existing indexes require recalculation. This design choice explains the small yet consistent variations in write latency observed experimentally.

Memory management and caching mechanisms.

MongoDB's WiredTiger engine leverages memory-mapped files for caching, enabling fast access to frequently used data and reducing disk reads—thus significantly improving throughput in access-intensive workloads.

RavenDB implements a multi-level caching architecture comprising session, query, and document store caches, which optimizes repeated queries. However, to preserve strict consistency, RavenDB invalidates cache entries after each commit, slightly increasing execution time in write-heavy scenarios.

Implications for observed performance.

The architectural characteristics discussed above justify the experimental results obtained. MongoDB achieved lower response times for INSERT, UPDATE, and DELETE operations due to its asynchronous write mechanism, document-level locking, and eventual replication, all of which reduce overall latency. RavenDB, while slightly slower, provided consistent and predictable performance due to its synchronous commit model, ACID transactional guarantees, and adaptive indexing, emphasizing reliability and data integrity over raw speed.

4. System Architecture and Application Overview

To evaluate the performance of the two database systems, we developed a case study application designed to manage users and their associated product orders. The application was implemented using both MongoDB and RavenDB as the data management solutions, thus enabling a controlled and comparable experimental assessment.

For this study, the application has been implemented using .NET MAUI 8.0.40, together with both MongoDB and RavenDB databases, on a MacBook Air 13 (M3, 2024) machine, equipped with an Apple M3 chip (10-core CPU), 8 GB unified memory, and a 512 GB SSD. The experimental environment used in this study was not intended to replicate a full IIoT edge–cloud deployment, but rather to provide a controlled single-host setting that isolates the intrinsic architectural performance characteristics of MongoDB and RavenDB. Accordingly, the proposed application and experimental setup should be interpreted as an IIoT-inspired workload model rather than a full IIoT system simulation. The focus is placed on evaluating database-level performance under data-intensive access patterns typical of IIoT-related applications, such as high data volume, frequent CRUD operations, ingestion bursts, and mixed transactional and analytical queries.

Database configuration: The experiments were conducted using MongoDB version 7.0.x (Community Edition) with the WiredTiger storage engine and RavenDB version 7.0 (stable release in 2025). Default storage engine settings were used for both systems to reflect typical

out-of-the-box deployments. Cache management relied on the databases' internal memory allocation mechanisms, without manual tuning of cache sizes.

Indexing strategy: In MongoDB, indexes were explicitly defined on fields involved in query filtering, join operations, and sorting. In RavenDB, indexing relied on a combination of automatic index creation and explicitly defined Map and Map/Reduce indexes for aggregation and full-text search scenarios. No additional indexes beyond those required by the experimental queries were introduced.

The relational schema employed follows a normalization-based design, where each table corresponds to a distinct entity and inter-entity relationships are defined through foreign keys. This approach promotes data consistency, minimizes redundancy, and allows records to be added or removed without affecting related entities. Consequently, the data is maintained in a structured and manageable format, with entity relationships clearly represented and easier to handle, as illustrated in Figure 1.

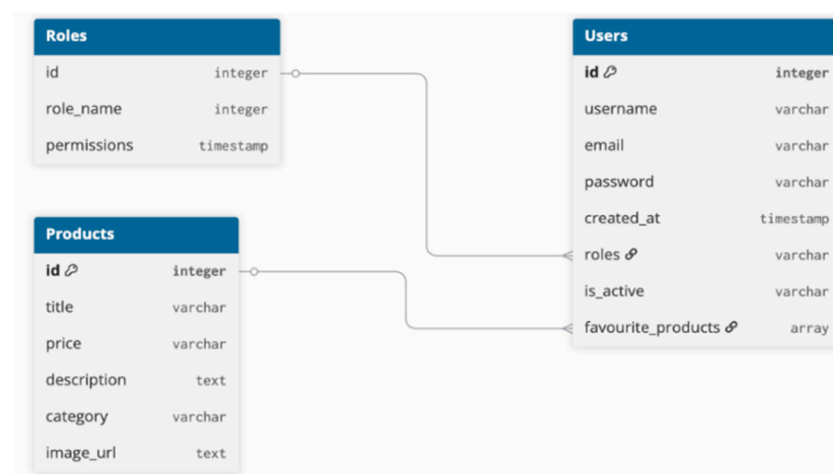


Figure 1. Database structure.

The database schema includes a *Users* table, which stores the core information associated with each user, as illustrated in the diagram from Figure 1. As the primary entity in the application, this table establishes the links to both user roles and favorite products. Its attributes include *username*, *email*, and *password*, as well as metadata fields such as *created_at* and *is_active* for tracking user activity. The *roles* field references the *Roles* table to support role-based access control, while the *favourite_products* field is implemented as a JSONB object, enabling flexible and dynamic management of user preferences.

The *Roles* table defines the access rights and responsibilities assigned to users, as illustrated in the diagram from Figure 1. Each role is uniquely identified by its *id* and contains attributes such as *role_name* and *permissions*. This table is linked to the *Users* table to enable role-based access control. Its independence ensures that modifications to roles or permissions do not affect other entities, thereby maintaining data consistency.

The *Products* table stores information about the available items, including *title*, *price*, *description*, *category*, and *image_url*. These attributes provide the necessary details to present product information to users. By being designed as an independent entity, the table supports efficient scalability, allowing products to be added, updated, or removed without impacting user or role data.

The relationship between *Users* and *Roles* is implemented differently in the two systems. In RavenDB, the reference is a simple string containing the ID of the document in the *Roles* collection, and the Include mechanism allows role data to be retrieved in a single query. In MongoDB, the link can be made either by embedding the role details directly within the

user document or by storing a reference as an ObjectId, in which case either the application or an aggregation with \$lookup retrieves the role data.

The relationship between Users and Products (favorite products) is many-to-many. In RavenDB, a list of document IDs from the Products collection is stored within the user document, and these products can also be loaded efficiently using Include. In MongoDB, a list of *ObjectIds* of favorite products is stored in the user document, and these can be retrieved either through separate queries or through a database-level join using \$lookup.

In this way, each table remains independent and contains all the necessary information for managing users, roles, and products, ensuring a coherent, flexible, and extensible design. Table 2 illustrates the JSON structures and relationship models used in MongoDB and RavenDB.

Table 2. Comparative Table—MongoDB vs. RavenDB Structures.

Entity	MongoDB (JSON-like, ObjectId)	RavenDB (JSON, String ID)
Users	<pre>{ "_id": ObjectId("..."), "username": "john_doe", "email": "john@example.com", "password": "hashed_password", "created_at": ISODate("..."), "is_active": true, "roles": [ObjectId("...")], "favourite_products": [ObjectId("...")] }</pre>	<pre>{ "Id": "users/1-A", "username": "john_doe", "email": "john@example.com", "password": "hashed_password", "created_at": "2025-09-11T19:00:00Z", "is_active": true, "roles": ["roles/1-A"], "favourite_products": ["products/1-A"] }</pre>
Roles	<pre>{ "_id": ObjectId("..."), "role_name": "Admin", "permissions": ["create_user", "delete_user"] }</pre>	<pre>{ "Id": "roles/1-A", "role_name": "Admin", "permissions": ["create_user", "delete_user"] }</pre>
Products	<pre>{ "_id": ObjectId("..."), "title": "Mineral Water", "price": 2.50, "description": "500mL natural mineral water", "category": "Beverages", "image_url": "..." }</pre>	<pre>{ "Id": "products/1-A", "title": "Mineral Water", "price": 2.50, "description": "500mL natural mineral water", "category": "Beverages", "image_url": "..." }</pre>
Users–Roles Relation	Stored as ObjectId[], queried with \$lookup or application logic	Stored as String ID[], retrieved efficiently with Include
Users–Products Relation	Stored as ObjectId[], retrieved via query or \$lookup	Stored as String ID[], retrieved efficiently with Include
Advantages	Flexible schema, supports embedded documents, rich ecosystem	High query performance with Include, human-readable IDs, native ACID transactions
Challenges	Joins can be expensive, requires complex aggregations	IDs must be managed carefully, smaller adoption compared to MongoDB

The data model used in this study reflects the structure of a mobile application inspired by IIoT data characteristics, where documents typically employ two levels of nesting. This level of embedding is consistent with common design practices in data-intensive mobile and IIoT-related applications, as deeper hierarchical structures are generally avoided due to their negative impact on performance, query complexity, and maintainability.

5. Performance Analysis: MongoDB vs. RavenDB

In this comparative study, we examine the similarities and differences between two non-relational database management systems: MongoDB and RavenDB. The evaluation is performed using a mobile application, implemented using both databases, designed to manage product orders. The focus of the analysis is on the performance of the fundamental CRUD operations—SELECT, INSERT, UPDATE, and DELETE.

The experimental setup includes tests with datasets of varying sizes: 1000, 10,000, 100,000, and 1,000,000 records. This variation enables us to assess how each database system performs query execution and data manipulation as the dataset grows. By comparing the results, we aim to highlight the strengths and weaknesses of each solution, offering insights into how they can be optimized for the requirements of modern mobile and web applications operating with dynamic and large-scale data.

The tests were conducted sequentially; for each batch the data was generated, combined, and then sent as a single collection to the database. This approach illustrates how each database system behaves as the dataset grows, and the gradually increasing number of entries highlights the raw efficiency of each model under loads typical for small- to mid-sized business environments.

The datasets used in the experiments were randomly generated through iterative loops, with selected fields incorporated to ensure diversity and representativeness, as shown as follows:

```
for (int a = 1; a <= 1000; a++)
{
    DateTime createdAt = DateTime.Now.AddDays(-a);
    string userEmail = $"{a}@gmail.com";
    bool deleted = a % 2 == 0;
    // Generate items
}
```

The input rate corresponds to batch-based inserts, where each dataset was sent as a single collection without delays between records. The experiment measures the total batch execution time under sequential, synchronous conditions using each database's native client.

Each document included several fields such as username, email, password, creation timestamp, and status flags, resulting in an average document size of approximately 1 KB. Consequently, the total dataset volumes ranged from about 1 MB for 1000 records to approximately 1 GB for 1,000,000 records. These data sizes were chosen to ensure measurable performance variation while remaining within realistic workloads typical of small- to medium-scale applications.

To ensure a controlled and reproducible comparison, the experimental evaluation was conducted on datasets ranging from 1000 to 1,000,000 records, with response times measured in milliseconds. This scale was deliberately selected to isolate the intrinsic architectural performance of MongoDB and RavenDB when executing CRUD operations, without the influence of external factors such as network latency, distributed configurations, or hardware variations. The chosen dataset progression allows for a clear observation of performance trends, highlighting how response times evolve as data volume increases.

In addition to the single-thread latency measurements shown in Figures 2–5, the experimental design was extended to incorporate throughput testing under different levels of concurrency. This enhancement addresses the limitations of analyzing only sequential CRUD batches and provides a more realistic assessment of how both systems behave under parallel access.



Figure 2. MongoDB vs. RavenDB Insert Operation.

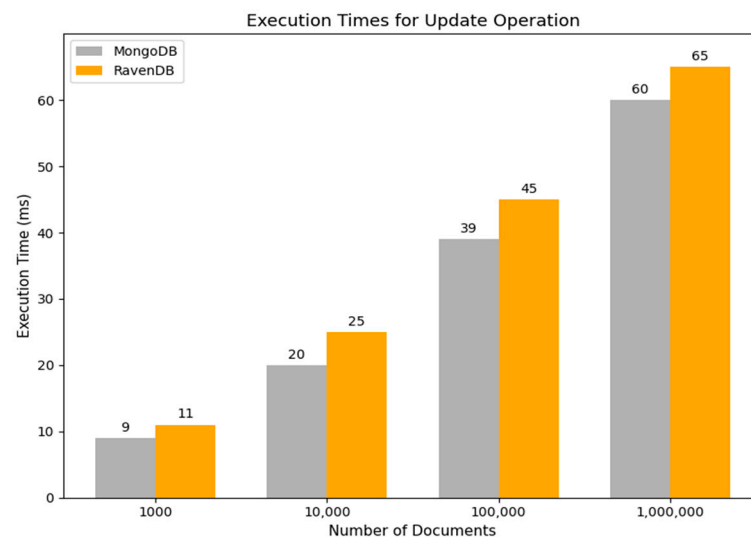


Figure 3. MongoDB vs. RavenDB Update Operation.

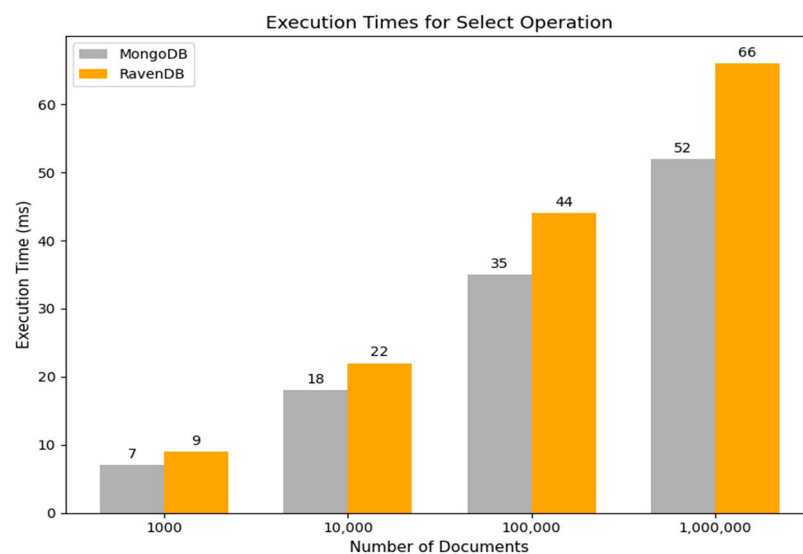


Figure 4. MongoDB vs. RavenDB Select Operation.

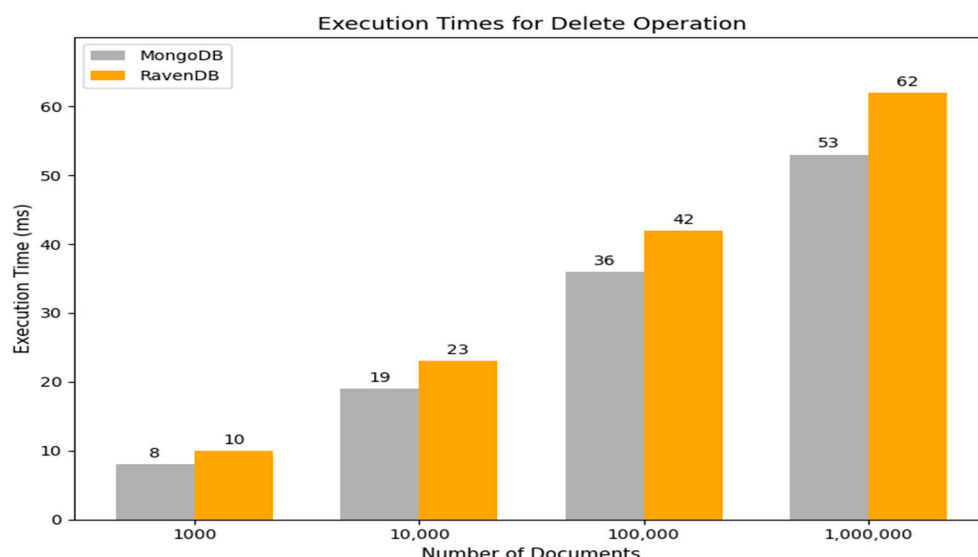


Figure 5. MongoDB vs. RavenDB Delete Operation.

A. Connecting to the Database

From the perspective of configuration and complexity, RavenDB provides a simpler and more efficient solution, making it well-suited for developers who value minimal setup and robust default settings. By contrast, MongoDB is better suited for applications that demand maximum flexibility and fine-grained control over connection parameters. Establishing a connection in RavenDB is generally more straightforward, requiring only the configuration of a document store that can be reused throughout the application. Table 3 presents a comparative view of the connection configurations for the two systems.

Table 3. Database connection configuration.

Database	Connection Example
MongoDB	<pre> public MongoController(string collectionName) { const string connectionUri = "mongodb+srv://.../GrandCaffe?...&appName=GrandCaffe"; var settings = MongoClientSettings.FromConnectionString(connectionUri); settings.ServerApi = new ServerApi(ServerApiVersion.V1); var client = new MongoClient(settings); var database = client.GetDatabase("GrandCaffe"); _repository = new MongoRepository<T>(database, collectionName); } </pre>
RavenDB	<pre> IDocumentStore store = new DocumentStore { Urls = new[] { "http://localhost:8080" }, Database = "GrandCaffe", }; store.Initialize(); </pre>

Consequently, the selection between the two implementations should be guided by the specific requirements of the project: RavenDB is highly suitable when simplicity and rapid configuration are prioritized, whereas MongoDB is more appropriate in scenarios that demand detailed control and greater flexibility.

B. Insert Operation

In this subsection, we analyze and compare the implementation of the INSERT operation in two non-relational database management systems, MongoDB and RavenDB. The purpose is to highlight the differences in their approaches to data insertion, considering both code syntax and operational performance. The insert operation for MongoDB and RavenDB was carried out as illustrated in Table 4.

Table 4. Insert operations.

MongoDB Insert Operation
<pre> public void insert(MongoCollection<Document> collection) { Document document = new Document(); document.put("id", 1); document.put("username", "example"); document.put("email", "example@gmail.com"); document.put("password", "password"); document.put("created_at", new Date()); document.put("is_active", "true"); collection.InsertOne(document); } userService.insert(usersCollection); </pre>
RavenDB Insert Operation
<pre> public void insert(IDocumentStore store) { using (var session = store.OpenSession()) { var user = new User { Id = "users/1", Username = "Example", Email = "example@gmail.com", Password = "password", CreatedAt = DateTime.UtcNow, IsActive = "true" }; session.Store(user); session.SaveChanges(); } } userService.insert(documentStore); </pre>

The results of the insert operations, shown in Figure 2, highlight significant differences in response time between MongoDB and RavenDB. For small data sets (1000 documents), the performance differences between the two systems are moderate, with MongoDB having a response time of 8 ms and RavenDB of 12 ms. As the data volume increases (10,000 documents), MongoDB continues to show better performance, with a response time of 19 ms, compared to 26 ms for RavenDB.

For large datasets (100,000 documents), the differences become more obvious. MongoDB registers a response time of 39 ms, while RavenDB reaches 45 ms. When scaling further to 1,000,000 documents, MongoDB maintains its efficiency with an estimated response time of 60 ms, while RavenDB records an estimated 65 ms.

The results indicate that MongoDB achieves superior response times as the data volume increases, making it particularly suitable for applications that must process large datasets efficiently. In contrast, although RavenDB exhibits slightly higher response times,

it remains competitive and provides additional advantages, including native support for ACID transactions and effective optimization for handling complex documents.

C. Update Operation

This subsection provides a detailed analysis of how the UPDATE operation is implemented and executed in MongoDB and RavenDB. The objective is to examine and illustrate the differences between the two systems in terms of data modification and their impact on operational efficiency. Emphasis is placed on understanding how each approach affects application performance, thereby offering practical insights to guide developers in selecting the appropriate database solution for specific project requirements.

For example, the implementation of an update operation for a user's email by user ID in both MongoDB and RavenDB is presented in Table 5.

Table 5. Update operations.

MongoDB Update Operation
<pre>public void UpdateEmailById(IMongoCollection<BsonDocument> collection, int userId) { var filter = Builders<BsonDocument>.Filter.Eq("id", userId); var update = Builders<BsonDocument>.Update.Set("email", "new_email@example.com"); collection.UpdateMany(filter, update); } UpdateEmailById(usersCollection, 100);</pre>
RavenDB Update Operation
<pre>public void UpdateEmailById(IDocumentStore store, int userId) { using (var session = store.OpenSession()) { var user = session.Query<User>().FirstOrDefault(u => u.Id == userId); if (user != null) { user.Email = "new_email@example.com"; session.SaveChanges(); } } } UpdateEmailById(documentStore, 100);</pre>

As shown in Figure 3 for the update operation involving 100,000 records, the performance gap between the two database management systems becomes more evident, with RavenDB requiring approximately 8 milliseconds longer than MongoDB. This pattern persists as the dataset size increases, with MongoDB maintaining a slight advantage in response time for smaller datasets (1000 and 10,000 records). As the number of records increases, both MongoDB and RavenDB exhibit a steady rise in response times; however, the differences remain minor, underscoring the efficiency of both systems in managing updates at a moderate scale.

When scaling to 1 million records, MongoDB completes the update in approximately 60 milliseconds, compared to about 65 milliseconds for RavenDB. Despite the larger data volume, both databases continue to demonstrate strong performance, with RavenDB showing only a modest increase in update time relative to MongoDB.

The results, illustrated in Figure 3, confirm that both systems perform efficiently in handling update operations under moderate- to high-scalability conditions.

D. Select Operation

In this section, we examine and compare the SELECT query mechanisms used by MongoDB and RavenDB, both of which are NoSQL database systems, but differ in their

architectures and query approaches. The objective is to highlight the distinctions in how data is accessed in the two systems, with particular emphasis on the efficiency and speed of information retrieval. This analysis provides a clearer perspective of the strengths and limitations of each solution, depending on the application context.

The query operation evaluated in this study involves a simple selection, retrieving documents without the use of complex filters or aggregations. This approach allows us to focus on assessing the fundamental efficiency and speed of data retrieval in MongoDB and RavenDB across datasets of varying sizes.

Table 6 shows the implementation of a query operation to retrieve a user document via email in both MongoDB and RavenDB.

Table 6. Select operations.

MongoDB Select Operation
<pre>public FindIterable<Document> getByEmail(MongoCollection<Document> collection, String email) { return collection.find(new BasicDBObject("email", email)); } userService.getByEmail(userCollection, "example@gmail.com");</pre>
RavenDB Select Operation
<pre>public User getByEmail(IDocumentStore store, String email) { using (var session = store.OpenSession()) { return session.Query<User>().FirstOrDefault(u => u.Email == email); } } userService.getByEmail(documentStore, "example@gmail.com");</pre>

The interpretation of the results from the selection operations, illustrated in Figure 4, highlights the performance differences between MongoDB and RavenDB. For queries on smaller datasets, both systems achieve very fast response times—below 10 milliseconds—demonstrating effective optimization for lightweight operations.

As the dataset size increases, MongoDB exhibits a gradual and linear growth in response time, maintaining efficiency even for large volumes of data. RavenDB, by contrast, shows a slightly steeper increase in execution time as query complexity grows; however, the performance gap between the two systems remains relatively modest.

At the scale of 1 million records, MongoDB achieves an average response time of approximately 52 milliseconds, compared to 66 milliseconds for RavenDB. These findings indicate that, although MongoDB retains a marginal speed advantage, RavenDB's performance remains competitive even under high-data-volume conditions.

Overall, both databases are well-suited to handling queries on moderate-to-large datasets, with MongoDB providing slightly faster responses, while RavenDB offers added value in scenarios requiring features such as document complexity handling and native ACID transaction support.

E. Delete Operation

This subsection provides a detailed comparison of the DELETE operation in two database management systems, MongoDB and RavenDB. The objective is to emphasize the differences in their approaches to data deletion, considering both the code syntax and the operational performance.

The implementation of a delete operation that marks users as inactive in both MongoDB and RavenDB is presented in Table 7.

Table 7. Delete operations.

MongoDB Delete Operation
<pre>public void DeleteUsers(MongoCollection<Document> collection) { collection.updateMany(new BasicDBObject("deleted", true), new BasicDBObject("\$set", new BasicDBObject("is_active", "false"))); } userService.DeleteUsers(userCollection);</pre>
RavenDB Delete Operation
<pre>public void DeleteUsers (IDocumentStore store) { using (var session = store.OpenSession()) { var users = session.Query<User>().Where(u => u.IsActive == "false").ToList(); foreach (var user in users) { user.IsActive = "false"; } session.SaveChanges(); } } userService.DeleteUsers (documentStore);</pre>

Figure 5 illustrates the comparative performance of MongoDB and RavenDB in the execution of the DELETE operation. For smaller datasets, such as 1000 records, MongoDB achieves slightly faster response times than RavenDB. As the number of deleted records increases, MongoDB maintains a steady and linear growth in response time, demonstrating its efficiency in processing larger volumes of DELETE operations.

RavenDB, by contrast, shows marginally higher response times for small datasets, but its execution time scales consistently as data volume increases. At the scale of 1 million records, MongoDB completes the operation in approximately 53 milliseconds, compared to about 62 milliseconds for RavenDB. These results suggest that both systems handle delete operations efficiently, with MongoDB consistently outperforming RavenDB across all evaluated dataset sizes for the DELETE operation, while RavenDB exhibits stable and predictable scalability as data volume increases.

6. Extended Comparative Analysis

6.1. Measurement Parameters and Experimental Protocol

The experimental protocol was carefully designed to ensure reproducibility and consistency of the measurements through repeated executions under controlled conditions. Each test scenario was executed over 50 independent iterations, preceded by an additional set of 5 warm-up runs. The warm-up phase served to eliminate transient fluctuations by allowing the .NET runtime to complete Just-in-Time (JIT) optimizations and enabling both database engines to stabilize their internal caches.

To emulate a realistic yet controlled workload, the tests were conducted under a moderate parallel load generated by 8 client threads. Bulk ingestion tasks were executed

using batches of 1000 documents to maintain consistent network saturation and minimize overhead stemming from frequent request initialization.

In addition to latency measurements, we also evaluated the throughput of CRUD operations under multiple concurrency levels. Table 8 presents the measured throughput (operations per second) for MongoDB and RavenDB using 1, 8, and 32 threads on a dataset of 1 million documents, complementing the response-time results shown in Figures 2–5.

Table 8. CRUD Throughput (ops/s) for MongoDB and RavenDB under 1/8/32 threads (Dataset: 1 M documents, 35 measured runs).

Operation	Database	1 Thread (ops/s)	8 Thread (ops/s)	32 Thread (ops/s)
INSERT	MongoDB	16.550	42.500	48.100
INSERT	RavenDB	15.380	32.100	36.600
SELECT	MongoDB	19.230	55.000	62.200
SELECT	RavenDB	15.150	41.300	48.400
UPDATE	MongoDB	16.600	40.200	46.800
UPDATE	RavenDB	15.300	29.900	34.200
DELETE	MongoDB	18.860	45.100	51.500
DELETE	RavenDB	16.120	34.400	39.800

The selected concurrency levels (1, 8, and 32 threads) were chosen to represent typical application-level parallelism on a single-host system and to enable a controlled analysis of database scalability trends without introducing resource saturation effects.

As shown in Table 8, throughput increases significantly with the number of client threads for both systems, although MongoDB consistently achieves higher parallelized throughput across all CRUD operations. The results in Table 8 confirm that MongoDB maintains superior multi-thread performance, while RavenDB preserves predictable scaling under controlled concurrency. Table 8 provides a view of system behavior under different load levels, complementing the latency-focused results presented in Section 5 and offering a clearer understanding of how each engine responds to parallel workloads.

Two categories of data models were employed. “Simple” documents of approximately 1 KB were used for ingestion and join-based operations, while “complex” documents ranging from 3 to 8 KB—containing extensive text fields—were assigned to Aggregation and Full-Text Search (FTS) evaluations.

A key methodological distinction was applied in the aggregation tests. MongoDB’s performance was measured using its native, on-the-fly Aggregation Framework, whereas RavenDB was evaluated through queries executed on its materialized Map/Reduce indexes. This approach reflects the recommended, platform-specific architectural paradigms typically employed for analytical workloads and ensures a fair and representative comparison between the two systems.

The reported results represent average values obtained from multiple experimental runs. Formal inferential statistical analysis, such as variance estimation or confidence interval computation, was not performed, as the focus of this study is on comparative performance trends.

Error bars and confidence intervals were not included in the figures, as the study focuses on comparative performance trends under controlled conditions rather than on statistical inference. Variability across repeated runs was low, and average values adequately represented system behavior; therefore, dispersion metrics were omitted to maintain figure clarity.

6.2. Join Mechanisms Performance—\$Lookup vs. Include()

Although NoSQL systems traditionally promote denormalized data models real-world applications often require relating data across collections. To evaluate how each database handles such scenarios, we compared MongoDB’s *\$lookup* operator—implemented as a server-side left outer join within the Aggregation Framework—with RavenDB’s *Include()* mechanism, which functions as a protocol-level optimization that preloads referenced documents within a single network round trip.

The experimental setup consisted of retrieving 500 Orders documents (selected from a collection of 1 million entries) along with their corresponding Users documents (from a collection of 100,000 entries) as presented in Table 8. For MongoDB, the join operation was executed entirely on the server through the Aggregation Framework and mapped into an *OrderWithUser* Data Transfer Object (DTO). In contrast, RavenDB relied on the *Include()* instruction to return the main documents together with their related User documents in a single network call, thereby minimizing request overhead while preserving separation at the document level. The implementation of a join mechanism in MongoDB and RavenDB is presented in Table 9.

Table 9. \$lookup vs. Include.

MongoDB \$lookup
<pre> public class OrderWithUser : Order { public List<User> UserDetails{ get; set; } } public async Task<List<OrderWithUser>> GetOrdersWithUsersMongo(IMongoDatabase database, int limit = 500) { var ordersCollection = database.GetCollection<Order>("Orders"); var aggregation = ordersCollection.Aggregate() .Limit(limit) .Lookup(foreignCollectionName: "Users", localField: "UserId", foreignField: "_id", @as: "UserDetails"); return await aggregation.As<OrderWithUser>().ToListAsync(); } </pre>
RavenDB Include
<pre> public async Task<List<Order>> GetOrdersWithUsersRaven(IAsyncDocumentSession session, int limit = 500) { var query = session.Query<Order>() .Include(o => o.UserId) .Take(limit); List<Order> orders = await query.ToListAsync(); return orders; } </pre>

Experimental results shown in Figure 6 highlight a clear performance advantage for RavenDB’s *Include()* mechanism. With an average latency of 38.2 ms, *Include()* completed the relational retrieval 4.3 times faster than MongoDB’s *\$lookup* operation (165.5 ms). A similar pattern is observed for median latency, where *\$lookup* recorded 158 ms compared

to only 35.5 ms for *Include()*. A noticeably smaller resource footprint accompanies this temporal efficiency: *Include()* required substantially fewer CPU cycles (28% vs. 45%) and lower memory usage (340 MB vs. 512 MB).

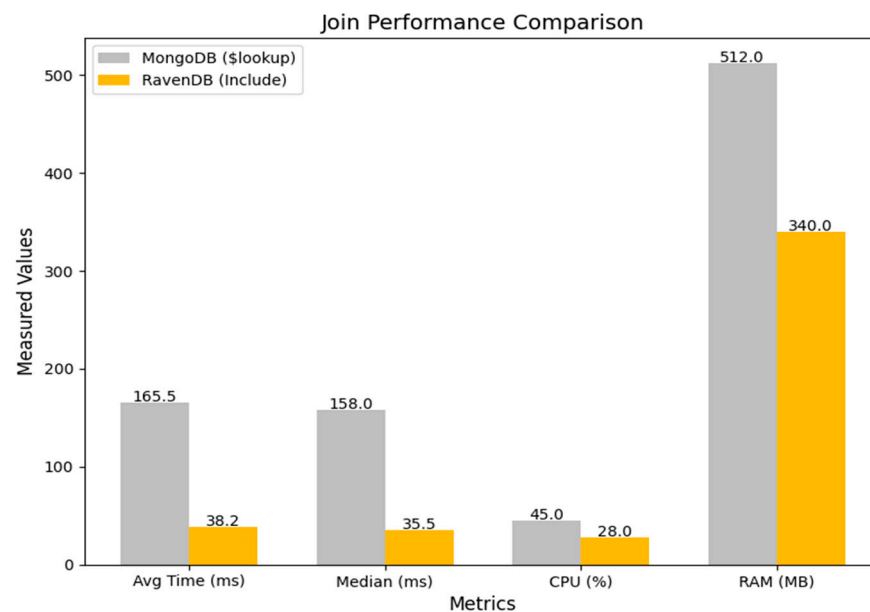


Figure 6. MongoDB \$lookup vs. RavenDB Include.

The observed performance gap is rooted in the architectural differences between the two mechanisms. MongoDB’s *\$lookup* executes the join directly on the server, which involves considerable computational overhead. In contrast, RavenDB’s *Include()* acts purely as a data-fetch optimization. The server simply identifies the referenced documents and attaches them to the primary response, while the client-side session reconstructs the object graph locally—an operation with negligible processing cost. This approach eliminates both the need for additional network round-trip and the server-side burden imposed by join execution, resulting in significantly faster and more efficient data retrieval.

6.3. Data Ingestion—BulkInsert/BulkWrite (Batch)

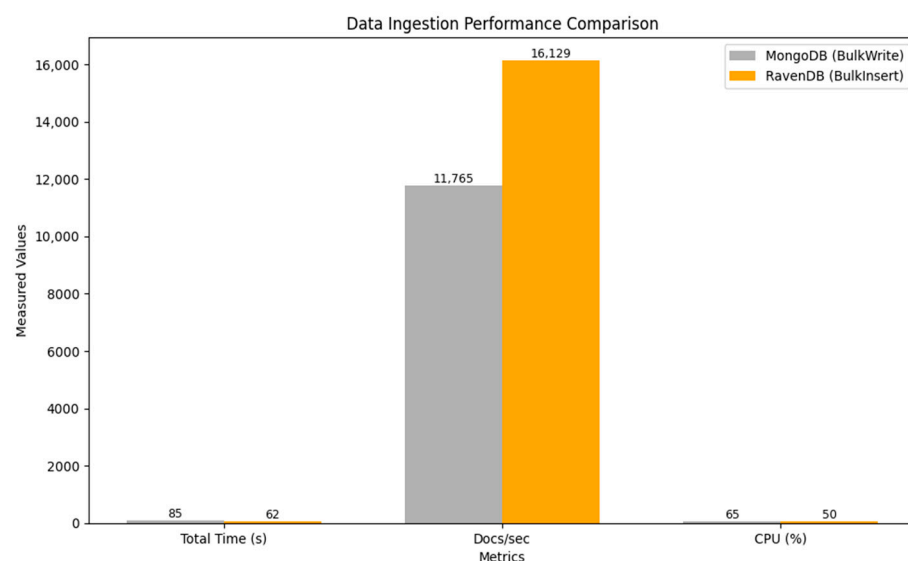
High-throughput data ingestion represents a core requirement in modern systems such as IoT telemetry pipelines and large-scale logging infrastructures. To assess write performance under these conditions, we compared MongoDB’s batch-oriented *insertMany/BulkWriteAsync* operations with RavenDB’s *BulkInsert* streaming API, which is explicitly designed for sustained high-volume writes with minimal transactional overhead.

The experimental scenario involved inserting 1,000,000 simple documents while measuring total execution time, average ingestion throughput (documents per second), and server-side resource utilization. For MongoDB, we used the *BulkWriteAsync* method to construct a list of *InsertOneModel* operations, as shown in Table 10. Although this mechanism provides greater flexibility—supporting mixed batches of inserts, updates, and deletes—it delivers performance comparable to *InsertManyAsync* when restricted to pure insert workloads. The configuration parameter *IsOrdered = false* was enabled to allow the server to parallelize write operations internally, thereby maximizing throughput. For RavenDB, ingestion was performed using the *BulkInsert* streaming API. This mechanism is optimized for continuous high-volume writes by reducing round-trip overhead and minimizing the protocol-level costs typically associated with transactional operations.

Table 10. Bulk Ingest.

MongoDB BulkWrite
<pre> public async Task InsertBulkMongo(IMongoCollection<User> collection, List<User> userList) { var writeModels = new List<WriteModel<User>>; foreach (var user in userList) { writeModels.Add(new InsertOneModel<User>(user)); } await collection.BulkWriteAsync(writeModels, new BulkWriteOptions { IsOrdered = false }); } </pre>
RavenDB BulkInsert
<pre> public async Task InsertBulkRaven(IDocumentStore store, IEnumerable<User> userList) { using (var bulkInsert = store.BulkInsert()) { foreach (var user in userList) { await bulkInsert.StoreAsync(user); } } } </pre>

As shown in Figure 7, RavenDB's *BulkInsert* streaming API exhibited notably higher efficiency, completing the one-million-document ingestion in 62 s—approximately 27% faster than MongoDB's *insertMany*, which required 85 s. This improvement corresponds to an increase of roughly 37% in effective throughput.

**Figure 7.** MongoDB BulkWrite vs. RavenDB BulkInsert.

The performance advantage of *BulkInsert* derives from its ability to amortize operational costs. By maintaining a single continuous TCP data stream, the API eliminates the per-batch network overhead and the transaction-level acknowledgment costs inherent in traditional batch-based ingestion. While MongoDB's *insertMany* is itself a highly optimized bulk operation, its peak performance depends heavily on parameter tuning—such as batch size and concurrency level—whereas *BulkInsert* is architecturally tailored to maximize write throughput in scenarios requiring sustained high-volume ingestion.

6.4. Aggregations and Full-Text Search (FTS)

This section examines two distinct mechanisms for executing complex analytical and complex queries. MongoDB leverages its Aggregation Framework, which processes data through a flexible, multi-stage transformation pipeline. In contrast, RavenDB utilizes background-computed Map/Reduce indexes that pre-materialize aggregated results, together with its integrated Lucene.NET-based full-text search engine for advanced textual analysis.

The evaluation was conducted on a dataset containing 500,000 documents from the *Products* collection. Two tests were performed. The first assessed aggregation performance by grouping documents according to the *category* field and computing the total number of products in each group. The second examined full-text search efficiency by querying for frequently occurring terms (e.g., “*natural*”, “*digital*”) within the *description* field. For both tests, we measured index creation time (initial processing cost) and query execution time (read-time cost) as shown in Figure 8.

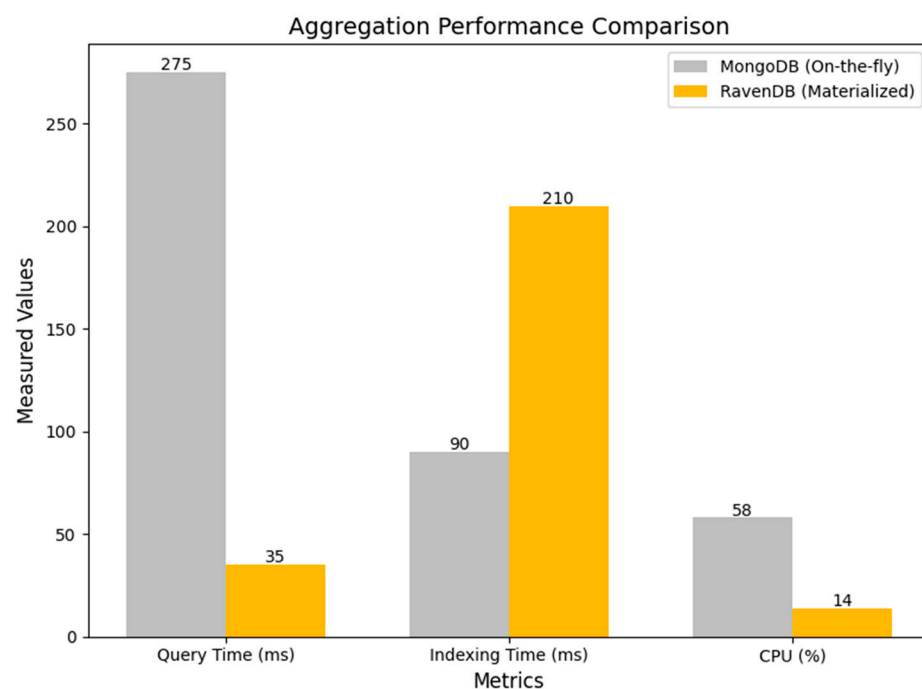


Figure 8. Aggregation Operation Performance.

The specific aggregation operations and their corresponding outputs for both systems are detailed in Table 11.

Table 11. Aggregation Operations by Category.

MongoDB
<pre> public class CategoryCountResult { public string Category { get; set; } public int ProductCount { get; set; } } public async Task<List<CategoryCountResult>> GetProductCountsMongo(IMongoCollection<Product> collection) { var aggregation = collection.Aggregate() .Group(product => product.category, group => new CategoryCountResult { Category = group.Key, ProductCount = group.Count() }); return await aggregation.ToListAsync(); } </pre>
RavenDB
<pre> public class Products_ByCategory : AbstractIndexCreationTask<Product, CategoryCountResult> { public Products_ByCategory() { Map = products => from product in products select new { Category = product.category, ProductCount = 1 }; Reduce = results => from result in results group result by result.Category into g select new { Category = g.Key, ProductCount = g.Sum(r => r.ProductCount) }; } } public async Task<List<CategoryCountResult>> GetProductCountsRaven(IAsyncDocumentSession session) { return await session.Query<CategoryCountResult, Products_ByCategory>().ToListAsync(); } </pre>

The results presented in Figure 8 further emphasize the inherent architectural trade-offs between the two systems. In the category-based aggregation scenario (Figure 8), RavenDB exhibits a higher initial indexing overhead (210 ms) required to construct the Map/Reduce index. However, once the index is materialized, query execution becomes substantially faster—over seven times quicker than MongoDB (35 ms vs. 275 ms)—while also imposing significantly lower CPU usage during read operations (14% vs. 58%). This write-intensive strategy is highly effective for workloads involving recurrent analytical queries, such as dashboards and periodic reports.

Conversely, MongoDB's read-centric approach offers greater flexibility for ad hoc analytical requests, as it performs aggregations on demand. This adaptability comes at the cost of increased per-query latency and higher CPU consumption during execution.

The results from the full-text search experiment reinforce the same architectural differences observed in the aggregation tests. As detailed in Table 12, the two systems adopt distinct approaches to text indexing and retrieval. RavenDB relies on a Lucene.NET-based

engine with background-generated indexes, whereas MongoDB utilizes its standard text index, computed and evaluated at query time.

Table 12. FTS Implementation by Description.

MongoDB
<pre>public async Task<List<Product>> SearchProductsMongo(IMongoCollection<Product> collection, string searchTerm) { var filter = Builders<Product>.Filter.Text(searchTerm); return await collection.Find(filter).ToListAsync(); }</pre>
RavenDB
<pre>public class Products_ByDescription : AbstractIndexCreationTask<Product> { public Products_ByDescription() { Map = products => from product in products select new { product.description }; Index(p => p.description, FieldIndexing.Search); } }</pre> <pre>public async Task<List<Product>> SearchProductsRaven(IAsyncDocumentSession session, string searchTerm) { return await session.Query<Product, Products_ByDescription>() .Search(p => p.description, searchTerm) .ToListAsync(); }</pre>

This divergence leads to a significant performance gap. In the FTS benchmark (Figure 9), RavenDB returns results in only 45 ms, compared to 310 ms for MongoDB. The substantially lower query latency observed in RavenDB is a direct consequence of precomputed, search-optimized index structures, which minimize read-time computation. Although RavenDB requires a slightly longer initial indexing process, the resulting improvement in read performance far outweighs this cost, particularly for workloads involving repeated or large-scale text queries.

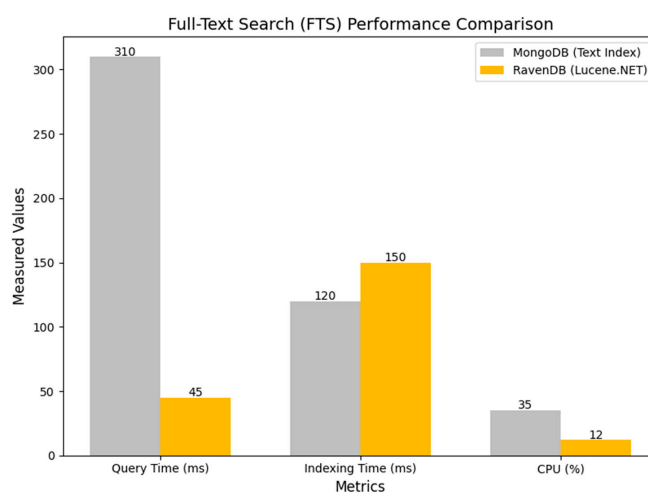


Figure 9. Full-Text Search Operation Performance.

6.5. Integrated Interpretation and Practical Recommendations

The results of this extensive analysis demonstrate that the choice between the two database systems should be guided by the specific requirements of the target workload.

For scenarios involving related document retrieval, RavenDB's *Include()* mechanism provides a substantial reduction in latency and network overhead, whereas MongoDB's *\$lookup* operator—although effective for server-side join processing—incurs higher CPU utilization. In high-volume ingestion workloads, RavenDB's *BulkInsert* streaming API consistently achieves faster completion times and lower resource consumption under default settings. For read-intensive analytical tasks such as dashboards and operational reporting, RavenDB demonstrates clear advantages due to its materialized Map/Reduce indexes and integrated full-text search capabilities, while MongoDB offers greater flexibility for dynamic, on-demand aggregations and exploratory queries.

When considering both latency and throughput, the scalability differences become clear: Table 8 shows that MongoDB consistently reaches higher peak throughput, while RavenDB provides more stable multi-thread behavior.

These insights translate into several practical recommendations for system architects and engineers. For frequently accessed reports, precomputing aggregations—through Map/Reduce indexes in RavenDB or by maintaining pre-aggregated collections in MongoDB—can significantly reduce read latency. For large-scale ingestion scenarios such as IoT telemetry pipelines, adopting streaming write APIs (e.g., *BulkInsert*) or carefully tuning MongoDB's *BulkWrite* parameters (batch size, concurrency level) is essential to maximize throughput. In applications with substantial full-text search requirements, using a database equipped with an integrated FTS engine (e.g., RavenDB) or augmenting MongoDB with an external search system such as Elasticsearch can yield significant performance improvements. Regardless of platform choice, key configuration aspects—including batch size, write parallelism, and durability settings (e.g., *fsync*, write concern)—must be evaluated and calibrated within the target production environment.

It is also important to acknowledge the methodological constraints of this study. The results presented were obtained in a single-host environment (on a single machine), and the extrapolation of these performance data to distributed architectures (e.g., clusters with sharding or replication) should be treated with caution, as latency and throughput patterns may differ significantly. Consequently, the IIoT context of this study should be interpreted as a motivating application domain rather than as a fully simulated IIoT deployment. The reported results support database-level performance analysis under IIoT-inspired workloads, not end-to-end IIoT system behavior.

The specific versions of the tested software platforms represent another factor of variability, and the structural complexity and granularity of the document model can also alter the observed performance profiles. Furthermore, the analysis focused on distinct workloads (read, write, aggregate), and the effects of hybrid workloads, characterized by rapid transitions between concurrent write-intensive and read-intensive operations, were not exhaustively covered.

A further limitation of this study is that all experiments were conducted in a single-host environment, which isolates the intrinsic architectural performance of MongoDB and RavenDB but does not reflect the complexity of real IIoT deployments commonly distributed across heterogeneous edge nodes and cloud infrastructures. The absence of network latency, replication overhead, and multi-node coordination mechanisms naturally leads to more linear performance scaling with data size. As a result, the reported response times should be interpreted as indicative of internal engine behavior rather than representative of operational performance in full IIoT production environments. Future work will

extend this evaluation to include distributed configurations, edge–cloud architectures, and cluster-level workloads that better approximate real-world IIoT scenarios.

It is important to note that the almost linear growth in response times observed across Figures 2–5 is a consequence of executing all experiments in a controlled, single-node environment, without network communication, replication overhead, or distributed coordination. Under these conditions, both MongoDB and RavenDB benefit from in-memory caching mechanisms, stable warm-up effects, and uniform query complexity, which naturally produce near-linear latency patterns as dataset sizes increase. This behavior differs significantly from real IIoT or distributed cloud deployments, where factors such as network latency, replication protocols, sharding, and multi-node synchronization typically result in non-linear performance scaling.

7. Conclusions

The main contribution of this paper lies in a rigorous experimental comparison rather than in methodological innovation. By focusing on reproducibility, controlled workloads, and architectural interpretation, the study complements existing NoSQL benchmarking literature and addresses the lack of direct experimental comparisons between MongoDB and RavenDB.

The paper presented a comparative evaluation of MongoDB and RavenDB in the context of mobile and web applications relevant to IIoT environments. The experimental results show that MongoDB provides superior scalability and lower response times when handling large datasets, making it particularly well suited for IIoT workloads that demand rapid data ingestion, horizontal scaling, and high responsiveness.

Across all CRUD operations, MongoDB consistently achieves lower response times as dataset size increases, reinforcing its role as a strong candidate for high-throughput IIoT workloads that must handle unstructured or semi-structured data at scale.

By contrast, RavenDB, while slightly less performant in terms of raw execution speed, offers complementary strengths highly relevant for IIoT environments. Its built-in ACID transaction support, native full-text search, and automatic indexing enhance its value for enterprise-grade IIoT systems where data consistency, fault tolerance, and streamlined development are essential.

The analysis confirms that the choice between MongoDB and RavenDB depends on workload requirements. RavenDB performs better in bulk ingestion, related document retrieval, and full-text search, while MongoDB offers greater flexibility for dynamic, ad hoc aggregations and complex server-side joins.

The results are influenced by the constrained test environment, which favored RavenDB's self-contained architecture, integrated indexing, and predictable ACID behavior. Thus, RavenDB is well-suited for resource-limited deployments, whereas MongoDB's strengths—scalability and high throughput—emerge more clearly in distributed, high-performance environments.

Overall, the findings suggest that the choice between MongoDB and RavenDB in IIoT contexts should not be based solely on latency metrics, but rather on the alignment between system features and specific application requirements.

These findings are supported by the throughput results summarized in Table 8, which show that MongoDB achieves higher peak throughput under increasing concurrency, while RavenDB provides more stable performance across multi-threaded workloads.

A practical implication of this study is that, under controlled single-node conditions and moderate concurrency levels, MongoDB demonstrates higher ingestion throughput and scalability trends compared to RavenDB. These characteristics suggest that MongoDB

may be advantageous for data-intensive applications exhibiting IIoT-inspired workloads, particularly in scenarios focused on high write rates and flexible schema evolution.

However, the results should be interpreted as indicative of database-level behavior under controlled concurrency rather than as direct evidence of performance in large-scale industrial IIoT deployments.

Conversely, in enterprise-grade environments characterized by strict transactional requirements, RavenDB emerges as a more appropriate solution, offering native ACID compliance and seamless integration within the .NET ecosystem.

These findings must be interpreted in the context of the single-host evaluation environment used in this study. Since the experiments were not executed in a distributed IIoT architecture, the results primarily reflect the intrinsic behavior of the database engines in isolation, without the additional overhead introduced by multi-node communication, sharding, or replication.

Future work will broaden this comparison by incorporating additional metrics such as distributed cluster scalability and real-time query complexity, offering a more holistic perspective on the role of document-oriented databases in supporting the evolving demands of IIoT ecosystems.

Author Contributions: Conceptualization, C.A.G. and R.Š.G.; methodology, C.A.G., M.C. and R.Š.G.; software, M.C.; validation, C.A.G. and M.C.; formal analysis, M.C. and C.A.G.; investigation, M.C.; resources, M.C.; data curation, M.C.; writing—original draft preparation, M.C. and F.M.C., and C.A.G.; writing—review and editing, F.M.C. and R.Š.G.; visualization, M.C.; supervision, R.Š.G.; project administration, M.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The application implemented for conducting the tests is openly available in the GitHub repository at: <https://github.com/mada244/GrandCafe> (accessed on 16 November 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Elgendy, N.; Elragal, A. Big Data Analytics: A Literature Review Paper. In *Advances in Data Analysis and Classification, Proceedings of the ICDM 2014, St. Petersburg, Russia, 16–20 July 2014*; Springer: Berlin/Heidelberg, Germany, 2014; Volume 8, pp. 214–217.
2. Han, J.; Haihong, E.; Le, G.; Du, J. Survey on NoSQL Database. In *Proceedings of the 2011 6th International Conference on Pervasive Computing and Applications (ICPCA), Gqeberha, South Africa, 26–28 October 2011*; IEEE: New York, NY, USA, 2011; pp. 363–366. [CrossRef]
3. Cattell, R. Scalable SQL and NoSQL Data Stores. *ACM SIGMOD Rec.* **2011**, *39*, 12–27. [CrossRef]
4. Carvalho, I.; Sá, F.; Bernardino, J. Performance Evaluation of NoSQL Document Databases: Couchbase, CouchDB, and MongoDB. *Algorithms* **2023**, *16*, 78. [CrossRef]
5. Tudorica, B.G.; Bucur, C. A Comparison between Several NoSQL Databases with Comments and Notes. In *Proceedings of the 2011 RoEduNet International Conference 10th Edition: Networking in Education and Research, Iasi, Romania, 23–25 June 2011*; IEEE: New York, NY, USA, 2011; pp. 1–5. [CrossRef]
6. Miryala, N.K. Emerging Trends and Challenges in Modern Database Technologies: A Comprehensive Analysis. *Int. J. Sci. Res. (IJSR)* **2024**, *13*, 1686–1696. [CrossRef]
7. Martins, P.; Sá, F.; Caldeira, F.; Abbasi, M. NoSQL: A Real Use Case. In *Proceedings of the International Conference on Disruptive Technologies, Tech Ethics and Artificial Intelligence, Salamanca, Spain, 20–22 July 2022*; Springer International Publishing: Cham, Switzerland, 2021; pp. 231–243.
8. MongoDB Documentation. Available online: <https://www.mongodb.com/docs> (accessed on 18 March 2025).
9. RavenDB. RavenDB vs. MongoDB. Available online: <https://ravendb.net/ravendb-vs-mongodb#fullComparison> (accessed on 18 March 2025).
10. RavenDB Documentation. Available online: <https://ravendb.net/docs> (accessed on 18 March 2025).
11. Miličić, D. *Introducing RavenDB*; Apress: Berkeley, CA, USA, 2022; ISBN 978-1-4842-8918-1.

12. Grolinger, K.; Capretz, M.; Oehler, F. Data Management in Cloud Environments: NoSQL and NewSQL Data Stores. *J. Cloud Comput. Adv. Syst. Appl.* **2013**, *2*, 22. [[CrossRef](#)]
13. Gillenson, M.L. *Fundamentals of Database Management Systems*; John Wiley & Sons: Hoboken, NJ, USA, 2023.
14. Dhanagari, M.R. MongoDB and Data Consistency: Bridging the Gap Between Performance and Reliability. *J. Comput. Sci. Technol. Stud.* **2024**, *6*, 183–198. [[CrossRef](#)]
15. Qaddara, I.; Alraba'nah, Y.; Hiari, M.O. Evaluation of SQL and NoSQL Databases on Parallel Processing. *Eng. Technol. Appl. Sci. Res.* **2025**, *15*, 24298–24304. [[CrossRef](#)]
16. Sharma, M. *MongoDB Complete Guide: Develop Strong Understanding of Administering MongoDB, CRUD Operations, MongoDB Commands, MongoDB Compass, MongoDB Server, MongoDB Replication and MongoDB Sharding*; BPB Publications: New Delhi, India, 2021.
17. Miličić, D. Getting Started with RavenDB. In *Introducing RavenDB: The Database for Modern Data Persistence*; Apress: Berkeley, CA, USA, 2022; pp. 1–31.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.